

Python 3 — String, Bit, Conversion & NumPy Cheatsheet

Built-in methods · Number conversions · NumPy string & bit ops · print-ready reference

String Manipulation

Method / Operator	Returns	Description	Example
Case conversion			
<code>str.upper()</code>	str	All cased chars → uppercase.	<code>"hello".upper()</code> → 'HELLO'
<code>str.lower()</code>	str	All cased chars → lowercase.	<code>"HELLO".lower()</code> → 'hello'
<code>str.title()</code>	str	First letter of every word capitalised.	<code>"hello world".title()</code> → 'Hello World'
<code>str.capitalize()</code>	str	First char upper, rest lower.	<code>"hello".capitalize()</code> → 'Hello'
<code>str.swapcase()</code>	str	Upper↔lower swap throughout.	<code>"Hello".swapcase()</code> → 'hELLO'
<code>str.casefold()</code>	str	Aggressive lowercase for caseless matching.	<code>"Straße".casefold()</code> → 'strasse'
Search & count			
<code>str.find(sub[, s, e])</code>	int	Lowest index of sub, or -1 if absent.	<code>"banana".find("an")</code> → 1 <code>"banana".find("an", 2)</code> → 3
<code>str.rfind(sub[, s, e])</code>	int	Highest (rightmost) index of sub, or -1.	<code>"banana".rfind("an")</code> → 3
<code>str.index(sub[, s, e])</code>	int	Like find() but raises ValueError if absent.	<code>"hello".index("l")</code> → 2
<code>str.rindex(sub[, s, e])</code>	int	Like rfind() but raises ValueError.	<code>"hello".rindex("l")</code> → 3
<code>str.count(sub[, s, e])</code>	int	Non-overlapping occurrences of sub.	<code>"banana".count("a")</code> → 3 <code>"aaa".count("aa")</code> → 1
<code>str.startswith(pfx[, s, e])</code>	bool	True if string begins with prefix (or tuple of prefixes).	<code>"hello".startswith("he")</code> → True
<code>str.endswith(sfx[, s, e])</code>	bool	True if string ends with suffix (or tuple).	<code>"hello".endswith("lo")</code> → True
Modify & pad			
<code>str.replace(old, new[, n])</code>	str	Replace all (or first n) occurrences of old with new.	<code>"aabbaa".replace("aa", "X")</code> → 'XbbX'
<code>str.strip([chars])</code>	str	Remove leading & trailing whitespace (or chars).	<code>" hi ".strip()</code> → 'hi'
<code>str.lstrip([chars])</code>	str	Strip leading characters only.	<code>" hi ".lstrip()</code> → 'hi '
<code>str.rstrip([chars])</code>	str	Strip trailing characters only.	<code>" hi ".rstrip()</code> → ' hi'
<code>str.removeprefix(pfx)</code>	str	Remove prefix if present. (Python 3.9+)	<code>"TestHook".removeprefix("Test")</code> → 'Hook'
<code>str.removesuffix(sfx)</code>	str	Remove suffix if present. (Python 3.9+)	<code>"MiscTests".removesuffix("Tests")</code> → 'Misc'
<code>str.zfill(width)</code>	str	Pad with leading zeros to total width.	<code>"42".zfill(5)</code> → '00042' <code>"-42".zfill(5)</code> → '-0042'
<code>str.center(w[, fill])</code>	str	Centre in a string of length w, padded with fill.	<code>"hi".center(8, "-")</code> → '---hi---
<code>str.ljust(w[, fill])</code>	str	Left-justify in a string of length w.	<code>"hi".ljust(6, ".")</code> → 'hi....'
<code>str.rjust(w[, fill])</code>	str	Right-justify in a string of length w.	<code>"hi".rjust(6, ".")</code> → '...hi'
<code>str.expandtabs([size])</code>	str	Replace tabs with spaces (default tabsize=8).	<code>"a\tb".expandtabs(4)</code> → 'a b'

Split, join & partition			
<code>str.split([sep[,max]])</code>	list	Split by sep (default: any whitespace). max limits splits.	<code>"a,b,c".split(",")</code> → ['a', 'b', 'c']
<code>str.rsplit([sep[,max]])</code>	list	Like split() but splits from the right.	<code>"a,b,c".rsplit(", ", 1)</code> → ['a, b', 'c']
<code>str.splitlines([keepends])</code>	list	Split at line boundaries (\n, \r\n, \r, ...).	<code>"a\nb\nc".splitlines()</code> → ['a', 'b', 'c']
<code>str.partition(sep)</code>	tuple	3-tuple (before, sep, after) at first occurrence.	<code>"user@host".partition("@")</code> → ('user', '@', 'host')
<code>str.rpartition(sep)</code>	tuple	3-tuple at last occurrence of sep.	<code>"a.b.c".rpartition(".")</code> → ('a.b', '.', 'c')
<code>str.join(iterable)</code>	str	Concatenate strings in iterable with str as separator.	<code>"", ".join(["a", "b", "c"])</code> → 'a, b, c'

Is* predicates			
<code>str.isalpha()</code>	bool	All chars alphabetic, non-empty.	<code>"hello".isalpha()</code> → True
<code>str.isdigit()</code>	bool	All chars decimal digits (includes ² ³ ...).	<code>"123".isdigit()</code> → True
<code>str.isnumeric()</code>	bool	All chars numeric (broader: fractions, roman numerals).	<code>"½".isnumeric()</code> → True
<code>str.isdecimal()</code>	bool	All chars decimal characters (strictest).	<code>"123".isdecimal()</code> → True
<code>str.isalnum()</code>	bool	All chars alphanumeric (letters or digits).	<code>"abc123".isalnum()</code> → True
<code>str.isspace()</code>	bool	All chars whitespace (space, tab, \n, ...).	<code>" \t\n".isspace()</code> → True
<code>str.isupper()</code>	bool	All cased chars are uppercase.	<code>"ABC".isupper()</code> → True
<code>str.islower()</code>	bool	All cased chars are lowercase.	<code>"abc".islower()</code> → True
<code>str.istitle()</code>	bool	String is titlecased.	<code>"Hello World".istitle()</code> → True
<code>str.isidentifier()</code>	bool	Valid Python identifier.	<code>"my_var".isidentifier()</code> → True
<code>str.isprintable()</code>	bool	All chars are printable.	<code>"hello!".isprintable()</code> → True
<code>str.isascii()</code>	bool	All chars are ASCII (0–127). Python 3.7+	<code>"hello".isascii()</code> → True

Formatting			
<code>str.format(*args, **kw)</code>	str	String formatting via {} replacement fields.	<code>"{} and {}".format("cats", "dogs")</code> → 'cats and dogs'
<code>f'...'</code>	str	f-string inline expression interpolation (fastest).	<code>f"{name} is {age}"</code> → 'Alice is 30' <code>f"{2**10:,}"</code> → '1,024'
<code>str % values</code>	str	Old-style printf-like formatting.	<code>"%.2f" % 3.14159</code> → '3.14'
<code>str.translate(table)</code>	str	Map chars through table from str.maketrans().	<code>t=str.maketrans("aeiou", "*****")</code> <code>"hello".translate(t)</code> → 'h*ll*'
<code>str.maketrans(x[,y[,z]])</code>	dict	Build translation table. z chars are deleted.	<code>str.maketrans("", "", "aeiou")</code> # delete vowels
<code>str.encode([enc[,err]])</code>	bytes	Encode string as bytes (default: utf-8).	<code>"café".encode("utf-8")</code> → b'caf\xcc\xa9'
<code>bytes.decode([enc[,err]])</code>	str	Decode bytes to string (default: utf-8).	<code>b'caf\xcc\xa9'.decode()</code> → 'café'

Bit Manipulation			
Method / Operator	Returns	Description	Example

Operators			
<code>x & y</code>	int	Bitwise AND — 1 only where both bits are 1.	<code>0b1010 & 0b1100</code> → 0b1000 (8)
<code>x y</code>	int	Bitwise OR — 1 where either bit is 1.	<code>0b1010 0b0101</code> → 0b1111 (15)
<code>x ^ y</code>	int	Bitwise XOR — 1 where bits differ. $x^x=0$, $x^0=x$	<code>0b1010 ^ 0b1100</code> → 0b0110 (6)
<code>~x</code>	int	Bitwise NOT — flips all bits. Equals $-(x+1)$.	<code>~5</code> → -6 <code>~0</code> → -1
<code>x << n</code>	int	Left shift — multiply x by 2^n .	<code>1 << 3</code> → 8 (1×2^3) <code>5 << 2</code> → 20 (5×4)
<code>x >> n</code>	int	Right shift — floor-divide x by 2^n .	<code>8 >> 1</code> → 4 <code>-8 >> 1</code> → -4 (sign preserved)
<code>x &= y</code> <code> = y</code> <code>^= y</code>	int	In-place bitwise assignment.	<code>x = 0b1111</code> <code>x &= 0b1010</code> → <code>x = 0b1010</code> (10)

int methods & builtins			
<code>int.bit_length()</code>	int	Bits needed to represent the integer (no sign/leading zeros).	<code>(255).bit_length()</code> → 8 <code>(0).bit_length()</code> → 0
<code>int.bit_count()</code>	int	Number of 1-bits (popcount). Python 3.10+	<code>(0b1011).bit_count()</code> → 3 <code>(255).bit_count()</code> → 8
<code>int.to_bytes(len, order)</code>	bytes	Return integer as bytes. order: 'big' or 'little'.	<code>(1024).to_bytes(2, "big")</code> → <code>b'\x04\x00'</code>
<code>int.from_bytes(b, order)</code>	int	Construct int from bytes. Class method. signed=True for negatives.	<code>int.from_bytes(b'\x04\x00', "big")</code> → 1024
<code>bin(x)</code>	str	Binary string prefixed '0b'.	<code>bin(10)</code> → '0b1010'
<code>hex(x)</code>	str	Hex string prefixed '0x'.	<code>hex(255)</code> → '0xff'
<code>oct(x)</code>	str	Octal string prefixed '0o'.	<code>oct(8)</code> → '0o10'
<code>int(x, base)</code>	int	Parse string as integer in given base (2-36). base=0 auto-detects.	<code>int("1010", 2)</code> → 10 <code>int("0xFF", 16)</code> → 255
<code>format(x, 'b'/'x'/'o')</code>	str	Format int as binary/hex/octal. Supports width & fill.	<code>format(10, '08b')</code> → '00001010' <code>format(255, 'x')</code> → 'ff'

Number Conversions			
Method / Operator	Returns	Description	Example
Integer conversions			
<code>int(x)</code>	int	Convert x to integer. Truncates floats; parses strings in base 10.	<code>int(3.9)</code> → 3 <code>int("42")</code> → 42 <code>int(True)</code> → 1
<code>int(x, base)</code>	int	Parse string x as integer in given base (2–36). base=0 auto-detects prefix.	<code>int("ff", 16)</code> → 255 <code>int("0b101", 0)</code> → 5 <code>int("0o17", 0)</code> → 15
<code>bin(x)</code>	str	Return binary string of integer x, prefixed '0b'.	<code>bin(10)</code> → '0b1010' <code>bin(-3)</code> → '-0b11'
<code>oct(x)</code>	str	Return octal string of integer x, prefixed '0o'.	<code>oct(8)</code> → '0o10' <code>oct(255)</code> → '0o377'
<code>hex(x)</code>	str	Return hex string of integer x, prefixed '0x' (lowercase).	<code>hex(255)</code> → '0xff' <code>hex(256)</code> → '0x100'
<code>format(x, spec)</code>	str	Format integer without prefix. Supports width, fill, sign.	<code>format(10, "b")</code> → '1010' <code>format(10, "08b")</code> → '00001010' <code>format(255, "X")</code> → 'FF'
<code>f"{x:b/o/x/X/d}"</code>	str	f-string format specs: b=binary, o=octal, x=hex lower, X=upper, d=decimal.	<code>f"{255:x}"</code> → 'ff' <code>f"{255:#010b}"</code> → '0b11111111'

Float conversions			
<code>float(x)</code>	float	Convert x to float. Accepts int, str, or bool.	<code>float(3)</code> → 3.0 <code>float("3.14")</code> → 3.14 <code>float("inf")</code> → inf
<code>round(x[, n])</code>	float/int	Round to n decimal places (default 0 → int). Uses banker's rounding.	<code>round(3.14159, 2)</code> → 3.14 <code>round(2.5)</code> → 2 # banker's rounding
<code>math.floor(x)</code>	int	Largest integer ≤ x.	<code>import math</code> <code>math.floor(3.9)</code> → 3 <code>math.floor(-3.1)</code> → -4
<code>math.ceil(x)</code>	int	Smallest integer ≥ x.	<code>math.ceil(3.1)</code> → 4 <code>math.ceil(-3.9)</code> → -3
<code>math.trunc(x)</code>	int	Truncate toward zero (same as int() for floats).	<code>math.trunc(3.9)</code> → 3 <code>math.trunc(-3.9)</code> → -3
<code>math.modf(x)</code>	tuple	Return (fractional, integer) parts of x, both as floats.	<code>math.modf(3.75)</code> → (0.75, 3.0) <code>math.modf(-3.75)</code> → (-0.75, -3.0)
<code>math.frexp(x)</code>	tuple	Return (mantissa, exponent) such that x = m * 2^e, 0.5 ≤ m < 1.	<code>math.frexp(8.0)</code> → (0.5, 4) <code>math.frexp(1.0)</code> → (0.5, 1)
<code>math.ldexp(m, e)</code>	float	Return m * 2^e (inverse of frexp).	<code>math.ldexp(0.5, 4)</code> → 8.0
<code>f"{x:.nf/.ne/.ng}"</code>	str	Format float: f=fixed, e=scientific, g=shorter of both.	<code>f"{3.14159:.2f}"</code> → '3.14' <code>f"{12345.6:.2e}"</code> → '1.23e+04' <code>f"{0.00012:.3g}"</code> → '0.00012'
<code>f"{x:,.2f}"</code>	str	Comma thousands separator with 2 decimal places.	<code>f"{1234567.89:,.2f}"</code> → '1,234,567.89'
<code>f"{x:+.2f}"</code>	str	Always show sign (+ or -).	<code>f"{3.14:+.2f}"</code> → '+3.14' <code>f"{-3.14:+.2f}"</code> → '-3.14'
<code>f"{x:%}"</code>	str	Format as percentage (multiplied by 100).	<code>f"{0.1756:.1%}"</code> → '17.6%'
Complex numbers			
<code>complex(re, im)</code>	complex x	Create complex number re + im*j.	<code>complex(3, 4)</code> → (3+4j) <code>complex(0, 1)</code> → 1j
<code>z.real / z.imag</code>	float	Access real or imaginary part of complex z.	<code>z = 3+4j</code> <code>z.real</code> → 3.0 <code>z.imag</code> → 4.0
<code>abs(z)</code>	float	Modulus (magnitude) of complex number.	<code>abs(3+4j)</code> → 5.0
<code>z.conjugate()</code>	complex x	Return the complex conjugate (negate imaginary part).	<code>(3+4j).conjugate()</code> → (3-4j)
<code>cmath.phase(z)</code>	float	Phase angle in radians (−π to π).	<code>import cmath</code> <code>cmath.phase(1+1j)</code> → 0.785... (π/4)
<code>cmath.polar(z)</code>	tuple	Return (r, phi) polar coordinates.	<code>cmath.polar(1+1j)</code> → (1.414..., 0.785...)
<code>cmath.rect(r, phi)</code>	complex x	Convert polar (r, phi) to complex.	<code>cmath.rect(1.414, 0.785)</code> → (1+1j) approx

Numeric base literals			
<code>0b...</code> / <code>0B...</code>	<code>int</code>	Binary integer literal.	<code>0b1010</code> → 10 <code>0B1111</code> → 15
<code>0o...</code> / <code>0O...</code>	<code>int</code>	Octal integer literal.	<code>0o17</code> → 15 <code>0o377</code> → 255
<code>0x...</code> / <code>0X...</code>	<code>int</code>	Hexadecimal integer literal. A–F case-insensitive.	<code>0xFF</code> → 255 <code>0x1A</code> → 26
<code>1_000_000</code>	<code>int</code>	Underscore digit separator for readability (any base).	<code>1_000_000</code> → 1000000 <code>0xFF_FF</code> → 65535
<code>3.14e2</code>	<code>float</code>	Scientific notation float literal.	<code>3.14e2</code> → 314.0 <code>1.5e-3</code> → 0.0015
<code>struct.pack(fmt, v)</code>	<code>bytes</code>	Pack value into bytes using format string.	<code>import struct</code> <code>struct.pack(">I", 256)</code> → <code>b'\x00\x00\x01\x00'</code>
<code>struct.unpack(fmt, b)</code>	<code>tuple</code>	Unpack bytes to Python values.	<code>struct.unpack(">I", b'\x00\x01')</code> # big-endian uint
Type checks & special values			
<code>isinstance(x, int/float/complex)</code>	<code>bool</code>	Check numeric type.	<code>isinstance(3, int)</code> → True <code>isinstance(3.0, float)</code> → True
<code>math.isnan(x)</code>	<code>bool</code>	True if x is NaN (not-a-number).	<code>math.isnan(float("nan"))</code> → True
<code>math.isinf(x)</code>	<code>bool</code>	True if x is positive or negative infinity.	<code>math.isinf(float("inf"))</code> → True
<code>math.isfinite(x)</code>	<code>bool</code>	True if x is neither NaN nor infinity.	<code>math.isfinite(3.14)</code> → True
<code>math.isclose(a,b,rel_tol)</code>	<code>bool</code>	True if a and b are close within tolerance.	<code>math.isclose(0.1+0.2, 0.3)</code> → True
<code>float('inf')</code> / <code>float('nan')</code>	<code>float</code>	Special float constants.	<code>float("inf")</code> → inf <code>float("-inf")</code> → -inf <code>float("nan")</code> → nan

NumPy — String Methods (numpy.char / np.strings)			
Method / Operator	Returns	Description	Example
String creation & conversion			
<code>np.array(..., dtype=str)</code>	<code>ndarray</code>	Create string array. Use dtype='U' for fixed-width Unicode.	<code>np.array(["hi", "world"])</code> → <code>array(['hi', 'world'])</code>
<code>np.char.encode(a, enc)</code>	<code>ndarray</code>	Encode each element to bytes using encoding.	<code>np.char.encode(arr, "utf-8")</code>
<code>np.char.decode(a, enc)</code>	<code>ndarray</code>	Decode each bytes element to str.	<code>np.char.decode(barr, "utf-8")</code>
Case & whitespace			
<code>np.char.upper(a)</code>	<code>ndarray</code>	Uppercase every string element.	<code>np.char.upper(["hi", "bye"])</code> → <code>['HI', 'BYE']</code>
<code>np.char.lower(a)</code>	<code>ndarray</code>	Lowercase every element.	<code>np.char.lower(["HI", "BYE"])</code> → <code>['hi', 'bye']</code>
<code>np.char.title(a)</code>	<code>ndarray</code>	Titlecase every element.	<code>np.char.title(["hello world"])</code> → <code>['Hello World']</code>
<code>np.char.capitalize(a)</code>	<code>ndarray</code>	Capitalise first char of each element.	<code>np.char.capitalize(["hello"])</code> → <code>['Hello']</code>
<code>np.char.swapcase(a)</code>	<code>ndarray</code>	Swap upper/lower case in each element.	<code>np.char.swapcase(["Hello"])</code> → <code>['hELLO']</code>
<code>np.char.strip(a[, chars])</code>	<code>ndarray</code>	Strip leading/trailing chars from each element.	<code>np.char.strip([" hi "])</code> → <code>['hi']</code>
<code>np.char.lstrip(a[, chars])</code>	<code>ndarray</code>	Strip leading chars.	<code>np.char.lstrip(["xxhi"], "x")</code> → <code>['hi']</code>
<code>np.char.rstrip(a[, chars])</code>	<code>ndarray</code>	Strip trailing chars.	<code>np.char.rstrip(["hixx"], "x")</code> → <code>['hi']</code>

Search & test			
<code>np.char.find(a, sub[,s,e])</code>	<code>ndarray</code>	Lowest index of sub in each element, -1 if absent.	<code>np.char.find(["banana"], "an")</code> → <code>[1]</code>
<code>np.char.rfind(a, sub[,s,e])</code>	<code>ndarray</code>	Highest index of sub in each element.	<code>np.char.rfind(["banana"], "an")</code> → <code>[3]</code>
<code>np.char.index(a, sub)</code>	<code>ndarray</code>	Like find but raises ValueError if absent.	<code>np.char.index(["hello"], "l")</code> → <code>[2]</code>
<code>np.char.count(a, sub[,s,e])</code>	<code>ndarray</code>	Non-overlapping occurrences of sub.	<code>np.char.count(["banana"], "a")</code> → <code>[3]</code>
<code>np.char.startswith(a, pfx)</code>	<code>ndarray</code>	Boolean array — True where element starts with pfx.	<code>np.char.startswith(["hi", "bye"], "h")</code> → <code>[True, False]</code>
<code>np.char.endswith(a, sfx)</code>	<code>ndarray</code>	Boolean array — True where element ends with sfx.	<code>np.char.endswith(["hi", "bye"], "e")</code> → <code>[False, True]</code>
<code>np.char.isalpha(a)</code>	<code>ndarray</code>	Element-wise isalpha().	<code>np.char.isalpha(["abc", "1"])</code> → <code>[True, False]</code>
<code>np.char.isdigit(a)</code>	<code>ndarray</code>	Element-wise isdigit().	<code>np.char.isdigit(["123", "a"])</code> → <code>[True, False]</code>
<code>np.char.isalnum(a)</code>	<code>ndarray</code>	Element-wise isalnum().	<code>np.char.isalnum(["a1", "!"])</code> → <code>[True, False]</code>
<code>np.char.isupper(a)</code>	<code>ndarray</code>	Element-wise isupper().	<code>np.char.isupper(["ABC", "Abc"])</code> → <code>[True, False]</code>
<code>np.char.islower(a)</code>	<code>ndarray</code>	Element-wise islower().	<code>np.char.islower(["abc", "Abc"])</code> → <code>[True, False]</code>
<code>np.char.isspace(a)</code>	<code>ndarray</code>	Element-wise isspace().	<code>np.char.isspace([" ", "hi"])</code> → <code>[True, False]</code>
Modify & join			
<code>np.char.replace(a, old, new[,n])</code>	<code>ndarray</code>	Element-wise replace().	<code>np.char.replace(["aab"], "a", "x")</code> → <code>['xxb']</code>
<code>np.char.add(a1, a2)</code>	<code>ndarray</code>	Element-wise string concatenation.	<code>np.char.add(["hello"], ["world"])</code> → <code>['helloworld']</code>
<code>np.char.multiply(a, n)</code>	<code>ndarray</code>	Element-wise string repetition.	<code>np.char.multiply(["ab"], 3)</code> → <code>['ababab']</code>
<code>np.char.center(a, w[, fill])</code>	<code>ndarray</code>	Element-wise center().	<code>np.char.center(["hi"], 8, "-")</code> → <code>['---hi---']</code>
<code>np.char.ljust(a, w[, fill])</code>	<code>ndarray</code>	Element-wise ljust().	<code>np.char.ljust(["hi"], 6, ".")</code> → <code>['hi....']</code>
<code>np.char.rjust(a, w[, fill])</code>	<code>ndarray</code>	Element-wise rjust().	<code>np.char.rjust(["hi"], 6, ".")</code> → <code>['...hi']</code>
<code>np.char.zfill(a, width)</code>	<code>ndarray</code>	Element-wise zfill().	<code>np.char.zfill(["42"], 5)</code> → <code>['00042']</code>
<code>np.char.join(sep, a)</code>	<code>ndarray</code>	Join characters of each element with sep.	<code>np.char.join("-", ["abc"])</code> → <code>['a-b-c']</code>
Split			
<code>np.char.split(a[, sep[, max]])</code>	<code>ndarray</code>	Element-wise split(). Returns object array of lists.	<code>np.char.split(["a,b,c"], ",")</code> → <code>[['a', 'b', 'c']]</code>
<code>np.char.rsplit(a[, sep[, max]])</code>	<code>ndarray</code>	Element-wise rsplit().	<code>np.char.rsplit(["a,b,c"], ", ", 1)</code> → <code>[['a,b', 'c']]</code>
<code>np.char.splitlines(a)</code>	<code>ndarray</code>	Element-wise splitlines().	<code>np.char.splitlines(["a\nb"])</code> → <code>[['a', 'b']]</code>
<code>np.char.partition(a, sep)</code>	<code>ndarray</code>	Element-wise partition(). Returns object array of 3-tuples.	<code>np.char.partition(["user@h"], "@")</code> → <code>[('user', '@', 'h')]</code>

NumPy — Bit Manipulation & Type Conversions			
Method / Operator	Returns	Description	Example

Bitwise ufuncs (element-wise)			
<code>np.bitwise_and(x, y)</code>	<code>ndarray</code>	Element-wise bitwise AND. Equivalent to $x \& y$.	<code>np.bitwise_and([0b1010], [0b1100])</code> → [8]
<code>np.bitwise_or(x, y)</code>	<code>ndarray</code>	Element-wise bitwise OR. Equivalent to $x y$.	<code>np.bitwise_or([0b1010], [0b0101])</code> → [15]
<code>np.bitwise_xor(x, y)</code>	<code>ndarray</code>	Element-wise bitwise XOR. Equivalent to $x \wedge y$.	<code>np.bitwise_xor([0b1010], [0b1100])</code> → [6]
<code>np.bitwise_not(x)</code>	<code>ndarray</code>	Element-wise bitwise NOT ($\sim x$). Also: <code>np.invert(x)</code>	<code>np.bitwise_not(np.array([5], dtype=np.int8))</code> → [-6]
<code>np.invert(x)</code>	<code>ndarray</code>	Alias for <code>bitwise_not</code> . Works on bool arrays too.	<code>np.invert(np.array([True, False]))</code> → [False, True]
<code>np.left_shift(x, n)</code>	<code>ndarray</code>	Element-wise left shift. Equivalent to $x \ll n$.	<code>np.left_shift([1, 2, 4], 2)</code> → [4, 8, 16]
<code>np.right_shift(x, n)</code>	<code>ndarray</code>	Element-wise right shift. Equivalent to $x \gg n$.	<code>np.right_shift([8, 16, 32], 2)</code> → [2, 4, 8]
<code>np.packbits(a[, axis])</code>	<code>ndarray</code>	Pack bits of a bool/uint8 array into uint8 bytes.	<code>np.packbits([1, 0, 1, 0, 1, 0, 1, 0])</code> → [170] (0xAA)
<code>np.unpackbits(a[, axis])</code>	<code>ndarray</code>	Unpack uint8 elements into bits.	<code>np.unpackbits(np.array([170], dtype=np.uint8))</code> → [1, 0, 1, 0, 1, 0, 1, 0]
<code>np.binary_repr(x[, width])</code>	<code>str</code>	Binary string of integer x. Negative → two's complement with width.	<code>np.binary_repr(10)</code> → '1010' <code>np.binary_repr(-5, 8)</code> → '11111011'
Type casting & dtype conversions			
<code>arr.astype(dtype)</code>	<code>ndarray</code>	Cast array to a new dtype. <code>copy=False</code> avoids copy when possible.	<code>a.astype(np.float64)</code> <code>a.astype(np.int32)</code> <code>a.astype(str)</code>
<code>np.int8/16/32/64(x)</code>	<code>number</code>	Scalar constructors for fixed-width integer types.	<code>np.int8(127)</code> → 127 <code>np.int64(10**18)</code> → 1000000000000000000
<code>np.uint8/16/32/64(x)</code>	<code>number</code>	Unsigned integer scalar constructors.	<code>np.uint8(255)</code> → 255 <code>np.uint8(256)</code> → 0 (overflow)
<code>np.float16/32/64(x)</code>	<code>number</code>	Floating-point scalar constructors.	<code>np.float16(3.14)</code> → 3.14 (reduced precision) <code>np.float64(1/3)</code> → 0.3333...
<code>np.complex64/128(x)</code>	<code>number</code>	Complex scalar constructors.	<code>np.complex128(3+4j)</code> → (3+4j)
<code>np.bool_(x)</code>	<code>number</code>	NumPy boolean scalar.	<code>np.bool_(0)</code> → False <code>np.bool_(42)</code> → True
<code>np.iinfo(dtype)</code>	<code>object</code>	Integer type info: min, max, bits.	<code>np.iinfo(np.int32).max</code> → 2147483647 <code>np.iinfo(np.uint8).max</code> → 255
<code>np.finfo(dtype)</code>	<code>object</code>	Float type info: min, max, eps, precision.	<code>np.finfo(np.float32).eps</code> → 1.1920929e-07
<code>arr.view(dtype)</code>	<code>ndarray</code>	Reinterpret memory as a different dtype (no copy).	<code>a=np.array([1], dtype=np.int32)</code> <code>a.view(np.float32)</code> → reinterpreted bytes
<code>np.frombuffer(buf, dtype)</code>	<code>ndarray</code>	Interpret bytes buffer as 1-D array.	<code>np.frombuffer(b'\xff\x00', dtype=np.uint8)</code> → [255, 0]
<code>arr.tobytes()</code>	<code>bytes</code>	Return raw bytes of array data.	<code>np.array([1, 2], dtype=np.uint8).tobytes()</code> → b'\x01\x02'
<code>np.frompyfunc(f, nin, nout)</code>	<code>ufunc</code>	Create a NumPy ufunc from a Python function.	<code>f = np.frompyfunc(bin, 1, 1)</code> <code>f(np.array([1, 2, 4]))</code> → ['0b1', '0b10', '0b100']

Numeric conversions in arrays			
<code>np.around(a[,n])</code>	<code>ndarray</code>	Round to n decimals (alias: <code>np.round_</code>).	<code>np.around([1.567,2.345],1) → [1.6,2.3]</code>
<code>np.floor(a)</code>	<code>ndarray</code>	Element-wise floor.	<code>np.floor([1.9,-1.1]) → [1.,-2.]</code>
<code>np.ceil(a)</code>	<code>ndarray</code>	Element-wise ceiling.	<code>np.ceil([1.1,-1.9]) → [2.,-1.]</code>
<code>np.trunc(a)</code>	<code>ndarray</code>	Element-wise truncation toward zero.	<code>np.trunc([1.9,-1.9]) → [1.,-1.]</code>
<code>np.fix(a)</code>	<code>ndarray</code>	Round toward zero (same as <code>trunc</code> , alias).	<code>np.fix([-2.5, 2.5]) → [-2., 2.]</code>
<code>np.real(a) / np.imag(a)</code>	<code>ndarray</code>	Real or imaginary part of complex array.	<code>np.real([1+2j,3+4j]) → [1.,3.]</code> <code>np.imag([1+2j,3+4j]) → [2.,4.]</code>
<code>np.abs(a) / np.absolute(a)</code>	<code>ndarray</code>	Absolute value / modulus of each element.	<code>np.abs([-3,4,-5]) → [3,4,5]</code>
<code>np.sign(a)</code>	<code>ndarray</code>	Sign: -1, 0, or +1 for each element.	<code>np.sign([-5,0,3]) → [-1, 0, 1]</code>
<code>np.nan_to_num(a)</code>	<code>ndarray</code>	Replace NaN with 0, inf with large finite number.	<code>np.nan_to_num(np.array([np.nan, np.inf])) → [0., 1.8e+308]</code>

Strings are immutable — all `str` methods return a new string. Integers have arbitrary precision. `str.removeprefix/removesuffix` → Python 3.9+; `int.bit_count` → Python 3.10+.
`numpy.char` functions work element-wise on string arrays. NumPy bitwise ufuncs operate on integer dtypes; results depend on dtype width.